

Total Marks: _____

Obtained Marks: _____

(LAB) Object Oriented Programming Techniques Project

Last date of Submission: 18th May, 2026

Submitted To: Muhammad Hasaan Mujtaba

M Hashaam Asad	25108258	BS (AI) 2C
-----------------------	-----------------	-------------------

***Instructions:** Copied or shown assignments will be marked zero. Late submissions are not entertained in any case.*

Project Title: Smart University Opening and Weather Based Notification System

1. Introduction:

The Smart University Opening and Weather-Based Notification System is a console-based application developed in C++ using Object Oriented Programming (OOP) concepts. The purpose of this system is to assist universities in making decisions regarding opening or closing campuses based on weather conditions.

In real-world scenarios, extreme weather such as heavy rain, storms, or high temperatures can affect the safety of students and staff. This system simulates an automated decision-making process that evaluates weather conditions and determines whether the university should remain open, closed, or operate with caution.

2. Objectives of the Project:

The main objectives of this project are:

- To develop a weather-based decision system using C++
- To apply Object Oriented Programming concepts in a real-world scenario
- To automate university opening/closing decisions based on weather conditions
- To design a simple and user-friendly menu-driven system
- To practice arrays, loops, and conditional statements in C++

3. System Overview:

The system collects weather data such as temperature, rain level, and storm level. Based on these inputs, it analyzes conditions and displays the university status.

The system provides three possible outputs:

- **University Open (Normal Conditions)**
- **University Open with Caution (Moderate Weather)**
- **University Closed (Severe Weather Conditions)**

The application runs in a loop and allows the user to store multiple weather records and check system status accordingly.

4. Class Structure:

The system is designed using two main classes:

i. Weather Class

This class is responsible for storing weather-related data.

Attributes:

- Temperature (float)
- Rain Level (int)
- Storm Level (int)

Functions:

- setWeather() → Takes weather input from user
- Getter functions → Retrieve stored weather values

ii. UniversitySystem Class:

This class manages all weather records and decision-making logic.

Attributes:

- Array of Weather objects
- Counter variable to track records

Functions:

- addWeather() → Stores new weather data
- checkStatus() → Determines university status
- showAll() → Displays all stored weather records

5. OOP Concepts Used:

This project is based on core Object Oriented Programming principles:

- **Encapsulation:** Data members are kept private and accessed through functions
- **Abstraction:** Internal decision logic is hidden from the user
- **Objects and Classes:** Used to represent weather and system entities
- **Modularity:** Program is divided into separate classes and functions
- **Reusability:** Functions can be used multiple times for different records

6. Implementation Details:

The system is implemented using standard C++ features:

- Fixed-size array is used to store multiple weather records
- cin is used for input and cout for output
- for loop is used to display all records
- if-else conditions are used to analyze weather status
- switch-case is used for menu handling
- Menu-driven interface improves user interaction

7. Working of the System:

The system works in the following steps:

1. User selects an option from the menu
2. Weather data is entered (temperature, rain, storm level)
3. System stores the data in an array
4. System analyzes latest weather record
5. Based on conditions:
 - Normal → University Open
 - Moderate → Open with caution
 - Severe → University Closed
6. User can also view all stored records

8. Decision Logic:

The system follows these rules:

- If storm level is severe OR rain is heavy OR temperature > 45°C → **University Closed**
- If rain or storm is moderate OR temperature > 38°C → **Open with Caution**
- Otherwise → **University Open**

9. Limitations:

- Uses fixed size array (limited storage)
- No real-time weather API integration
- No file handling (data is not saved permanently)
- No graphical interface (console-based only)

Code:

```
#include <iostream>
```

```
#include <string>
```

```
using namespace std;
```

```
class Weather
```

```
{
```

```
private:
```

```
    float temperature;
```

```
    int rainLevel;
```

```
    int stormLevel;
```

```
    string date;
```

```
public:
```

```
    void setWeather()
```

```
    {
```

```
        cin.ignore();
```

```
        cout << "Enter Date: " << endl;
```

```
        getline(cin, date);
```

```
        cout << "Enter Temperature in Celsius: " << endl;
```

```
        cin >> temperature;
```

```
        cout << "Enter Rain Level (0 = No Rain, 1 = Light Rain, 2 = Heavy Rain): " << endl;
```

```
        cin >> rainLevel;
```

```
        cout << "Enter Storm Level (0 = No Storm, 1 = Mild Storm, 2 = Severe Storm): " << endl;
```

```
        cin >> stormLevel;
```

```
    }
```

```
    void displayWeather()
```

```
        cout << "Date: " << date << endl;
        cout << "Temperature: " << temperature << " C" << endl;
        cout << "Rain Level: " << rainLevel << endl;
        cout << "Storm Level: " << stormLevel << endl;
    }

    float getTemperature()
    {
        return temperature;
    }

    int getRainLevel()
    {
        return rainLevel;
    }

    int getStormLevel()
    {
        return stormLevel;
    }

    string getDate()
    {
        return date;
    }
};

class UniversitySystem
{
private:
    Weather weatherRecords[100];
    int totalRecords;

public:
```

UniversitySystem()

{

totalRecords = 0;

}

void addWeatherRecord()

{

if (totalRecords < 100)

{

weatherRecords[totalRecords].setWeather();

totalRecords++;

cout << "Weather Record Added Successfully!" << endl;

}

else

{

cout << "Storage Full!" << endl;

}

}

void displayAllRecords()

{

if (totalRecords == 0)

{

cout << "No Weather Records Found!" << endl;

return;

}

for (int i = 0; i < totalRecords; i++)

{

cout << "===== RECORD " << i + 1 << " =====" << endl;

weatherRecords[i].displayWeather();

cout << "===== " << endl;

```
void checkUniversityStatus()
```

```
{
```

```
    if (totalRecords == 0)
```

```
    {
```

```
        cout << "No Weather Data Available!" << endl;
```

```
        return;
```

```
    }
```

```
    Weather latest = weatherRecords[totalRecords - 1];
```

```
    cout << "===== UNIVERSITY STATUS REPORT =====" << endl;
```

```
    latest.displayWeather();
```

```
    if (latest.getStormLevel() == 2 || latest.getRainLevel() == 2 || latest.getTemperature() > 45)
```

```
    {
```

```
        cout << "University Status: CLOSED" << endl;
```

```
        cout << "Notification: Severe weather conditions detected." << endl;
```

```
        cout << "Students and staff are advised to stay at home." << endl;
```

```
    }
```

```
    else if (latest.getStormLevel() == 1 || latest.getRainLevel() == 1 || latest.getTemperature() >
```

```
38)
```

```
    {
```

```
        cout << "University Status: OPEN WITH CAUTION" << endl;
```

```
        cout << "Notification: Moderate weather conditions detected." << endl;
```

```
        cout << "Students should travel carefully." << endl;
```

```
    }
```

```
    else
```

```
    {
```

```
        cout << "University Status: OPEN" << endl;
```

```
        cout << "Notification: Weather conditions are normal." << endl;
```

```
    }
```



```
        cout << "===== " << endl;
    }

    void searchRecord()
    {
        string searchDate;
        bool found = false;

        cin.ignore();

        cout << "Enter Date to Search Record: " << endl;
        getline(cin, searchDate);

        for (int i = 0; i < totalRecords; i++)
        {
            if (weatherRecords[i].getDate() == searchDate)
            {
                cout << "Record Found!" << endl;

                weatherRecords[i].displayWeather();

                found = true;
                break;
            }
        }

        if (!found)
        {
            cout << "Record Not Found!" << endl;
        }
    }
};

int main()
```

UniversitySystem system;

int choice;

do

{

cout << endl;

cout << "===== SMART UNIVERSITY OPENING SYSTEM =====" << endl;

cout << "1. Add Weather Record" << endl;

cout << "2. Display All Records" << endl;

cout << "3. Check University Status" << endl;

cout << "4. Search Weather Record" << endl;

cout << "5. Exit" << endl;

cout << "Enter Your Choice: " << endl;

cin >> choice;

switch (choice)

{

case 1:

system.addWeatherRecord();

break;

case 2:

system.displayAllRecords();

break;

case 3:

system.checkUniversityStatus();

break;

case 4:

system.searchRecord();

break;

case 5:

```
cout << "Exiting System..." << endl;
```

```
break;
```

default:

```
cout << "Invalid Choice!" << endl;
```

```
}
```

```
} while (choice != 5);
```

```
return 0;
```

```
}
```

Output:

D:\BS (AI) 2C\OOP\Untitled1.exe

```
===== SMART UNIVERSITY OPENING SYSTEM =====
1. Add Weather Record
2. Display All Records
3. Check University Status
4. Search Weather Record
5. Exit
Enter Your Choice:
1
Enter Date:
18 May
Enter Temperature in Celsius:
24
Enter Rain Level (0 = No Rain, 1 = Light Rain, 2 = Heavy Rain):
2
Enter Storm Level (0 = No Storm, 1 = Mild Storm, 2 = Severe Storm):
1
Weather Record Added Successfully!

===== SMART UNIVERSITY OPENING SYSTEM =====
1. Add Weather Record
2. Display All Records
3. Check University Status
4. Search Weather Record
5. Exit
Enter Your Choice:
3
===== UNIVERSITY STATUS REPORT =====
Date: 18 May
Temperature: 24 C
Rain Level: 2
Storm Level: 1
University Status: CLOSED
Notification: Severe weather conditions detected.
Students and staff are advised to stay at home.
=====

===== SMART UNIVERSITY OPENING SYSTEM =====
1. Add Weather Record
2. Display All Records
3. Check University Status
4. Search Weather Record
5. Exit
Enter Your Choice:
5
Exiting System...

-----
Process exited after 51.52 seconds with return value 0
Press any key to continue . . .
```

10. Future Improvements:

The system can be improved by:

- Adding file handling for permanent weather storage
- Integrating real-time weather APIs
- Adding SMS/email notification system
- Creating a mobile or web-based version
- Improving prediction using AI-based models

Conclusion:

The Smart University Opening and Weather-Based Notification System is an efficient C++ application that demonstrates how programming can be used for real-world decision making. It successfully applies Object Oriented Programming concepts to simulate an automated university management system based on weather conditions.

This project helps in understanding how conditions, logic, and data structures can be combined to create intelligent systems.

THE END